Laboratory 5

Unbalanced parse trees and interpreter of postfix expressions

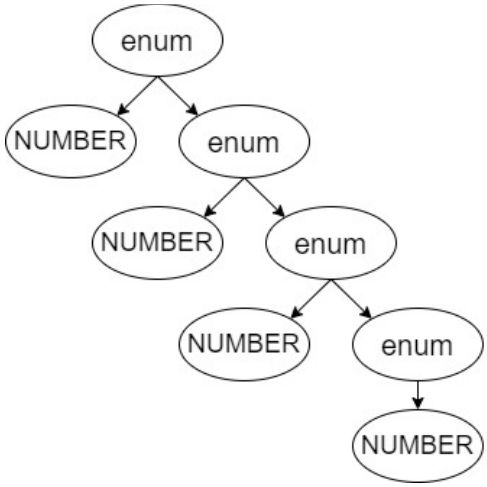
In the third laboratory we learned how symbols are stored on the stack and how semantic values are processed and propagated (using the pseudo-variables dollars) bottom-up in a parse tree. In the case of arithmetic expressions, only integers were stored on the value stack. After a reduction, they were substituted with integers. A reduction was happening when the right hand-side of a production was reduced/substituted with the left hand-side.

In the fourth laboratory we also worked with lists, case in which we had to define a %union with a member for integers and a member for lists. On the value stack both numbers and lists would coexist, so the stack had to be homogenous, with elements of the same type, union. This union would act like a wrapper that confers a common form for both numbers and lists. Although when reading a token from the input, it gets on the value stack as an integer, after a reduction (when an aggregated syntactic structure is recognized in the input), then an aggregated value gets to be stored on the value stack. In this case, the aggregated data structure is the list.

Therefore, we can imagine how the parse tree looks like when reading and creating a list. At each step, an integer element from a list is pushed on the value stack. When there is an element OR an element and a list on the value stack, they are reduced to a single list. The internal construction of a list, as a data structure, is described by the **enum** rule from the production set (see lab 4). The rule can be right recursive or left recursive. In the first case, elements are introduced in front of the list, so the parse tree is unbalanced on the right side. In the second case, elements are introduced at the end of the list, so the tree is unbalanced on the left side. Introducing an element at the end of a list is an inefficient method, because at each step we need to iterate through the whole list.

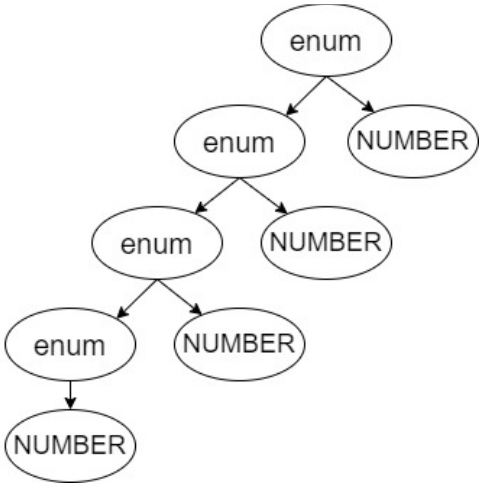
Right recursion:

```
enum -> NUMBER enum
      | NUMBER
```



Left recursion:

```
enum -> enum NUMBER
      | NUMBER
```



In the first case, all elements read so far from the list are stored on the value stack. After pushing the last element, it begins to pop elements one by one from the stack and to insert them in the list, starting from the last one (the rightmost) and ending with the first one (the leftmost). So, the elements are added to the final list in order, from right to left, by simply inserting them step by step in front of a partial list. In the second case, only the element to be introduced in the list at the current step is stored on the stack.

Let us take as an example the construction of list **'(1 2 3 4)**. In the next table we can see how the value stack evolves at the construction of a list right recursively or left recursively.

Right recursive	Left recursive
Value stack: 1 1 2 1 2 3 1 2 3 4 1 2 3 1 2 1	Value stack: 1 2 3 4
List construction: 4 3 4 2 3 4 1 2 3 4	List construction: 1 1 2 1 2 3 1 2 3 4

The next challenge is for us to create a lexico-syntactic analyzer, in order to transform arithmetic expressions from an infix to a postfix format.

Examples:

1 + 2 => 1 2 +
(a – 5 + c) * 2 => a 5 – c + 2 *
a / 2 + – (a – 5 + c) * 2 ^ b => a 2 / a 5 – c + – 2 b ^ * +

We observe that the input can contain variables, besides numbers and operators. Considering that we cannot calculate expressions with variables, all we can do is to change their form. So, we can store them as strings. In this case, the semantic communication channel between the lexical analyzer and the syntactic parser, **yylval**, must accept strings, therefore it can be represented as a %union with a member for strings.

In this exercise it is very important to carefully set the priorities of operations, in order to ensure a correct rearrangement of operations. We recall from the third laboratory that priorities of operations are specified in the declarations section of a .y file, using %left, %right, or %nonassoc.

The order in which priorities are declared shows the importance of operations: the first one has the lowest priority, and the last one has the highest priority. If an operation is %left associative, then a series of operations having the same priorities will be treated from left to right. If it is %right associative, then the series of operations will be treated from right to left. There are situations in which we need to define particular priorities. For example, if we treat negative numbers, their minus (unary minus) doesn't have the same priority as the subtraction minus (binary minus) and is not treated the same. So, we will define a special priority for the unary minus of negative numbers.

```
%left '+' '-'
%left '*' '/'
%left UNARYMINUS
%right '^'
```

In the set of productions, we must specify that the minus of negative numbers has a special priority. By "%prec UNARYMINUS" the precedence given by the generic '-' is overwritten. We already know the precedence of a rule is important in shift-reduce conflicts, when the precedence of a rule to reduce is compared to the precedence of the symbol to be shifted.

```
expr : expr '+' expr
     | expr '-' expr
     | expr '*' expr
     | expr '/' expr
     | expr '^' expr
     | '-' expr %prec UNARYMINUS
     | '(' expr ')'
     | NUMBER
     | VAR
     ;
```

Proposed exercises:

1. Create a lexico-syntactic analyzer that reads postfix arithmetic expressions and transforms them in prefix form. For example: 1 2 + => + 1 2
2. Create a lexico-syntactic analyzer that reads postfix arithmetic expressions without variables, only with numbers, and outputs the result of the computed expression.
   For example: 1 2 + => 3
   10 5 – 3 + 2 * => 16